



Soporte ACP

Utilice OpenCode en cualquier editor compatible con ACP.

OpenCode admite el Agent Client Protocol o (ACP), lo que le permite usarlo directamente en editores e IDE compatibles.

CONSEJO

Para obtener una lista de editores y herramientas compatibles con ACP, consulte el informe de progreso de ACP.

ACP es un protocolo abierto que estandariza la comunicación entre editores de código y agentes de codificación de IA.

Configuración

Para usar OpenCode a través de ACP, configure su editor para ejecutar el comando ``opencode acp``.

El comando inicia OpenCode como un subprocesso compatible con ACP que se comunica con su editor a través de JSON-RPC a través de stdio.

A continuación se muestran ejemplos de editores populares que admiten ACP.

Zed

Agregue a su configuración Zed (``~/config/zed/settings.json``):

```
~/ .config/zed/settings.json
```

```
{
  "agent_servers": {
    "OpenCode": {
      "command": "opencode",
      "args": ["acp"]
    }
  }
}
```

Para abrirlo, use la acción ``agent: new thread`` en la Paleta de comandos.

También puedes vincular un atajo de teclado editando tu ``keymap.json``:

```
keymap.json
```

```
[
  {
    "bindings": {
      "cmd-alt-o": [
        "agent::NewExternalAgentThread",
        {
          "agent": {
            "custom": {
              "name": "OpenCode",
              "command": {
                "command": "opencode",
                "args": ["acp"]
              }
            }
          }
        }
      ]
    }
  }
]
```

JetBrains IDEs

Agregue a su JetBrains IDE `acp.json` de acuerdo con la documentación:

`acp.json`

```
{
  "agent_servers": {
    "OpenCode": {
      "command": "/absolute/path/bin/opencode",
      "args": ["acp"]
    }
  }
}
```

Para abrirlo, use el nuevo agente 'OpenCode' en el selector de agentes de AI Chat.

Avante.nvim

Agregue a su configuración Avante.nvim:

```
{
  acp_providers = {
    ["opencode"] = {
      command = "opencode",
      args = { "acp" }
    }
  }
}
```

Si necesita pasar variables de entorno:

```
{
  acp_providers = {
    ["opencode"] = {
      command = "opencode",
      args = { "acp" },
      env = {
        OPENCODE_API_KEY = os.getenv("OPENCODE_API_KEY")
      }
    }
  }
}
```

CodeCompanion.nvim

Para usar OpenCode como agente ACP en CodeCompanion.nvim, agregue lo siguiente a su configuración de Neovim:

```
require("codecompanion").setup({
  interactions = {
    chat = {
      adapter = {
        name = "opencode",
        model = "claude-sonnet-4",
      },
    },
  },
})
```

Esta configuración configura CodeCompanion para usar OpenCode como agente ACP para el chat.

Si necesita pasar variables de entorno (como ``OPENCODE_API_KEY``), consulte Configuración de adaptadores: variables de entorno en la documentación de CodeCompanion.nvim para obtener detalles completos.

Soporte

OpenCode funciona igual a través de ACP que en la terminal. Todas las funciones son compatibles:

❗ NOTA

Algunos comandos de barra integrados como `/undo`` y `/redo`` no son compatibles actualmente.

Herramientas integradas (operaciones de archivos, comandos de terminal, etc.)

Herramientas personalizadas y comandos de barra

Servidores MCP configurados en su configuración OpenCode

Reglas específicas del proyecto de `AGENTS.md``

Formateadores y linters personalizados

Sistema de agentes y permisos.

 [Edita esta página](#)

© Anomaly

 [Found a bug? Open an issue](#)

Última actualización: 21 may 2026

 [Join our Discord community](#)

 [Español](#) ▼